ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

Faculty of Information Technology

WEB TECHNOLOGIES AND API SERVICES

Midterm Examination: Project Work Report

Production Management System

Case study: **Le Mieux Ltd**

A web application for manufacturing, inventory and HR management in a leather factory, built with Django and AdminLTE

Item	Details
Course	Web Technologies and API Services
Lecturer	Kundan Kumar
Technologies	Python, Django (back end); HTML, CSS, Bootstrap, AdminLTE (front end)
Group	Group 13
Submission date	18 October 2026

Group Members

No.	Full name	Registration number
1	Iradukunda Muhire Bertin	20251MBI046
2	Isaac Nkusi	20251MBI020
3	Rwabukanga Jean de Dieu	20251MBI010

Executive Summary

Le Mieux Ltd is a leather manufacturing company that produces handcrafted shoes, belts, bags and folders. Its production planning, stock records and staff records are currently kept on paper and in separate spreadsheets, which makes it hard to see who is busy, what a product really costs to make, and whether there is enough material in the warehouse before work starts.

This report presents the **Production Management System**, a database-driven web application that our group built with the Django framework to solve these problems. The system brings production, inventory and human resources (HR) together in one place. It is organised as three Django apps (`inventory`, `production` and `employees`) with seven related models, and it uses the AdminLTE template to give managers a clear dashboard.

Its key capabilities are:

- **Cost prediction:** each product has a Bill of Materials (BOM), from which the system calculates the predicted production cost and profit margin.
- **Stock protection:** a production task is accepted only if the warehouse holds enough raw material; stock is then deducted, and refunded automatically when a task is reduced or deleted.
- **Workload visibility:** a live dashboard shows recent tasks, overdue alerts and the number of active tasks per artisan.
- **Secure access:** full CRUD with confirmation before delete, login-protected management pages and the Django Admin site.

The sections that follow describe the problem, the system architecture, the database design (including an ERD that shows each model's attributes and operations), the URL routing, templates, validation rules, security, the challenges we faced and how to run the project.

Table of Contents

1. Introduction..... 5
1.1 Background..... 5
1.2 Problem Statement..... 5
1.3 Proposed Solution..... 5
1.4 Objectives..... 5
1.5 Main Features..... 5
1.6 Users and Use Cases..... 6
2. System Architecture and Project Structure..... 8
2.1 Technology Stack..... 8
2.2 Architecture Overview..... 8
2.3 Project Structure..... 9
2.4 Key Configuration..... 10
3. Database Design..... 11
3.1 Models and Relationships..... 11
3.2 Entity Relationship Diagram (ERD)..... 11
3.3 Model Operations (Methods)..... 13
3.4 Cardinality..... 13
3.5 Referential Integrity (on_delete Rules)..... 14
4. URL Routing and Views..... 15
4.1 Request Cycle Example..... 15
5. Templates and AdminLTE Integration..... 17
6. CRUD Operations and Validation..... 18
6.1 Core Operations..... 18
6.2 Advanced Validation (clean() Overrides)..... 19
7. Django Admin, Authentication and Authorisation..... 22
8. Challenges and Solutions..... 24
9. Conclusion and Future Work..... 24
Appendix A: How to Run the Project..... 25
Appendix B: Diagram Source Files..... 25

List of Figures

Figure 1: Use case diagram of the Production Management System....6
Figure 2: System architecture (Django MVT with three apps)....9
Figure 3: Entity Relationship Diagram with attributes and operations (designed in Dia)....12
Figure 4: Request cycle: from the URL to the rendered response....16
Figure 5: AdminLTE dashboard: summary cards, recent tasks table and Active Workload radar....17
Figure 6: Product creation form with the Bill of Materials (BOM) inline formset....18
Figure 7: Product detail page showing the BOM, predicted cost and profit margin....19
Figure 8: Delete confirmation page (confirm_delete.html)....19
Figure 9: Inventory protection and automatic stock deduction / refund logic....20
Figure 10: Validation error shown when warehouse stock is insufficient for a new task....21
Figure 11: Login page of the application....22
Figure 12: Django Admin site: registered models with list display, search and filters....23
Figure 13: HR roster of artisans with live contract and leave status....23

List of Tables

Table 1: Main features of the system....5
Table 2: User roles and permissions....7
Table 3: Technologies used....8
Table 4: Project parts and their responsibilities....9
Table 5: Database models, main fields and relationships....11
Table 6: Operations of each model....13
Table 7: Cardinality of the relationships....13
Table 8: Deletion rules and their effect....14
Table 9: Main URL patterns and the views that handle them....15
Table 10: Template components and AdminLTE integration....17
Table 11: CRUD operations implemented....18
Table 12: Validation and stock rules....20
Table 13: Django Admin and authentication features....22
Table 14: Demonstration superuser account....22
Table 15: Main challenges and how they were solved....24
Table 16: Dia source files....25

1. Introduction

1.1 Background

Le Mieux Ltd turns raw materials such as leather, thread, polish, buckles and zippers into finished leather goods. Every product follows a recipe of materials, every production order is carried out by an artisan, and every artisan has a contract and may request leave. These three areas (production, inventory and HR) depend on each other, but at present they are managed separately.

1.2 Problem Statement

Because the information is spread across paper records and spreadsheets, the company faces three main problems:

- **Uneven workload.** The HR team cannot easily see how many active tasks each artisan has, so some artisans are overloaded while others are idle.
- **Unknown costs and margins.** Production managers cannot reliably predict the cost of making a product or the profit it will earn.
- **Stock shortages during production.** Tasks are assigned without first checking the warehouse, so materials such as leather or buckles run out in the middle of production.

1.3 Proposed Solution

We developed the Production Management System, a web application built with Django. It centralises HR management, inventory tracking and the production workflow in a single database. The AdminLTE dashboard gives managers an immediate split view of recent production tasks and the current workload of every artisan.

1.4 Objectives

The **general objective** is to provide Le Mieux Ltd with one reliable web system for planning and tracking production. The **specific objectives** are to:

1. Record products, raw materials and the Bill of Materials that links them.
2. Predict production cost and profit margin automatically from the BOM.
3. Prevent production tasks from being created when stock is insufficient, and keep stock levels accurate.
4. Manage employees, their contracts and their leave requests.
5. Show the active workload of each artisan on a dashboard.
6. Protect management functions with authentication and authorisation.

1.5 Main Features

Table 1: *Main features of the system*

Feature	Description
Active workload dashboard	Summary cards, overdue-task alerts and tables that show recent tasks next to the number of active assignments per employee.

Feature	Description
BOM and cost prediction	Products are linked to their raw materials through inline formsets. The system calculates the predicted production cost and profit margin automatically.
Automated stock management	When a production task is created, the required BOM materials are checked against warehouse stock and deducted. Tasks are rejected if stock is insufficient.
HR management	A roster of artisans showing department, current contract status and leave requests in real time.
Full CRUD and workflow rules	Secure create, read, update and delete operations. Workflow rules are enforced; for example, a task cannot be marked "Completed" unless the produced quantity equals the target quantity.

1.6 Users and Use Cases

The system has three types of users. **Managers** (HR and production staff) maintain all business data. **Artisans** can log in and view information but cannot change it. The **Administrator** (superuser) can manage every record through the Django Admin site. Figure 1 shows what each user can do.

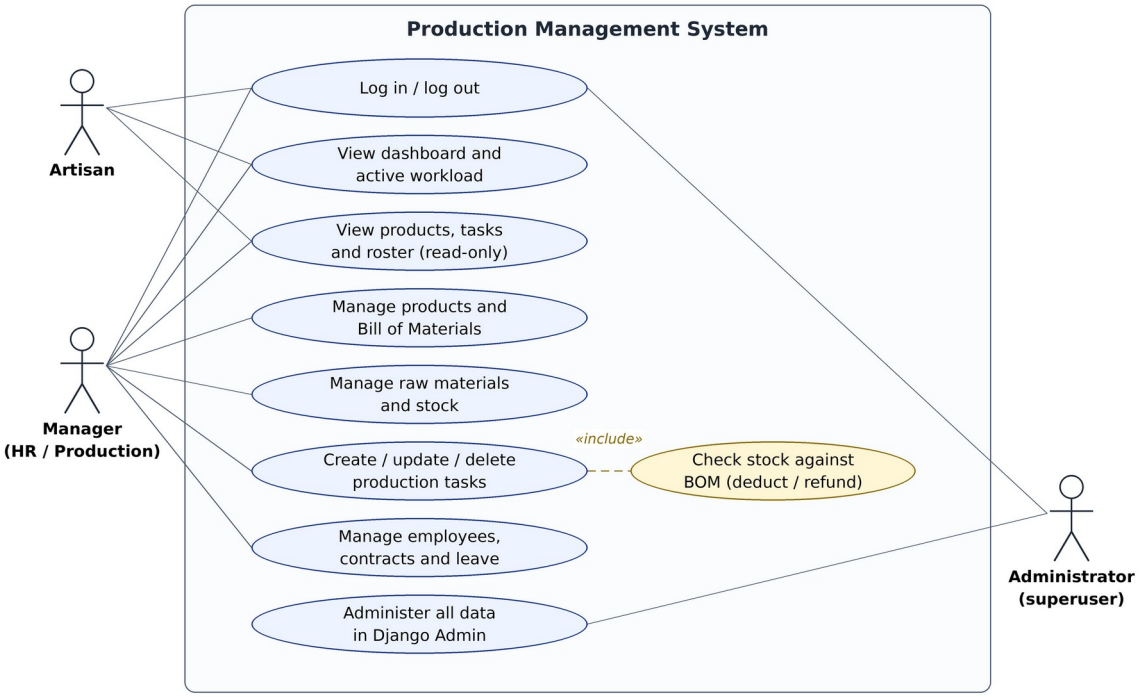


Figure 1: *Use case diagram of the Production Management System*

Table 2: *User roles and permissions*

Role	Can do	Cannot do
Manager (HR / Production)	Create, update and delete products, BOM lines, raw materials, production tasks, employees, contracts and leave requests; view the dashboard.	Access the Django Admin site (unless also given staff rights).
Artisan	Log in, view the dashboard, products, tasks and the roster.	Create, edit or delete any record (view-only).
Administrator (superuser)	Everything a manager can do, plus full data management in the Django Admin site (/admin/).	–

2. System Architecture and Project Structure

2.1 Technology Stack

Table 3: *Technologies used*

Layer	Technology	Role in the project
Back end	Python, Django	Models, views, forms, URL routing, ORM, authentication and admin.
Front end	HTML, CSS, Bootstrap	Page structure, styling and responsive grid layout.
UI template	AdminLTE, Font Awesome	Dashboard layout, sidebar, navigation bar, cards and icons.
Database	SQLite (Django default)	Stores the seven models; created through Django migrations.
Security	Django auth (LoginView, LogoutView, @login*required*, @*user*passes_test)	Login, logout and role-based protection of management pages.

2.2 Architecture Overview

The application follows Django's **Model–View–Template (MVT)** pattern. The browser sends an HTTP request; the project-level `urls.py` passes it to the correct app with `include()`; the app's view uses its models (through the Django ORM) and forms, and returns an HTML page rendered from a template that extends the shared AdminLTE layout. Figure 2 shows how these parts fit together.

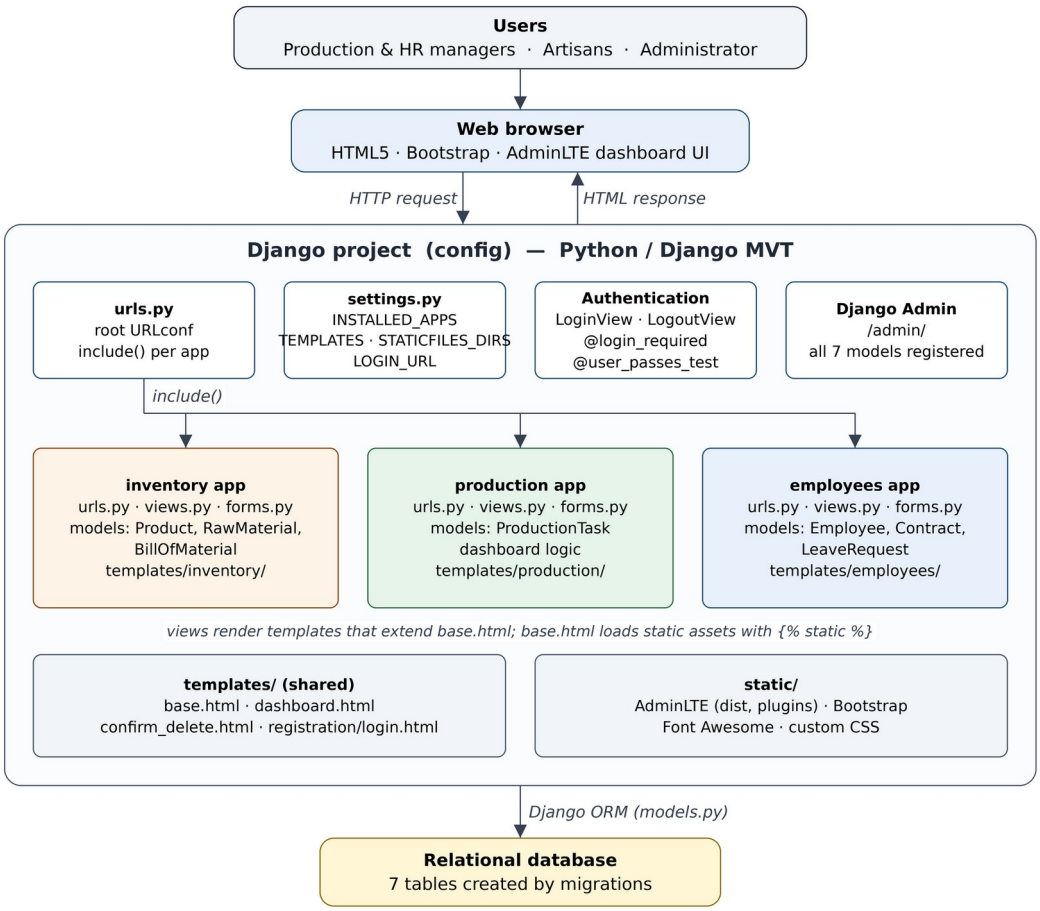


Figure 2: *System architecture (Django MVT with three apps)*

2.3 Project Structure

The project is divided into one configuration package (`config`) and three apps. Each app has its own `models.py`, `views.py`, `urls.py`, `forms.py` and `templates/<app>/` folder, which keeps every business area independent and easy to maintain.

Table 4: *Project parts and their responsibilities*

Part	Responsibility	Main files
config (project)	Settings, project-level URLs, WSGI/ASGI entry points and the dashboard view.	`settings.py`, `urls.py`, `wsgi.py`
inventory (app 1)	Manages Product, RawMaterial and BillOfMaterial.	`models.py`, `views.py`, `urls.py`, `forms.py`, `templates/inventory/`
production (app 2)	Manages ProductionTask and the dashboard logic.	`models.py`, `views.py`, `urls.py`, `forms.py`, `templates/production/`
employees (app 3)	Manages Employee, Contract and LeaveRequest.	`models.py`, `views.py`, `urls.py`, `forms.py`, `templates/employees/`

Part	Responsibility	Main files
templates/	Shared base layout, authentication pages and the AdminLTE dashboard.	`base.html`, `dashboard.html`, `confirm_delete.html`
static/	AdminLTE CSS and JavaScript, Bootstrap and Font Awesome.	`adminlte/dist/`, `adminlte/plugins/`

2.4 Key Configuration

The following settings in `config/settings.py` connect the three apps, the shared templates folder and the static files, and define where unauthenticated users are sent:

```
INSTALLED_APPS = [..., 'inventory', 'production', 'employees']
TEMPLATES = [{'DIRS': [BASE_DIR / 'templates'], ...}]
STATICFILES_DIRS = [BASE_DIR / 'static']
LOGIN_URL = 'login'
```

3. Database Design

The database contains **seven related models**, more than the minimum of three required. The design uses several one-to-many (ForeignKey) relationships and one many-to-many relationship, which is implemented through an intermediate ("through") model, BillOfMaterial.

3.1 Models and Relationships

Table 5: *Database models, main fields and relationships*

Model (app)	Main fields	Relationships
RawMaterial (inventory)	code (unique), name, unit, quantity*in*stock, unit_cost	Used in many products through BillOfMaterial.
Product (inventory)	code (unique), name, category, unit_price	1:N to ProductionTask; M:N to RawMaterial through BillOfMaterial.
BillOfMaterial (inventory)	quantity*per*unit	FK to Product (CASCADE); FK to RawMaterial (PROTECT).
Employee (employees)	first*name*, *last*name, department, is_active	1:N to Contract, LeaveRequest and ProductionTask.
Contract (employees)	contract*type*, *start*date, end*date*, *is*active	FK to Employee (CASCADE).
LeaveRequest (employees)	leave*type*, *start*date, end_date, status	FK to Employee (CASCADE).
ProductionTask (production)	task*id (unique)*, *title*, *target*quantity, produced_quantity, status	FK to Product (PROTECT); FK to Employee (SET_NULL).

3.2 Entity Relationship Diagram (ERD)

Figure 3 shows the ERD, drawn in Dia as UML classes. Each entity has three compartments: the **name** (with the owning Django app as a «stereotype»), the **attributes** (database fields) and the **operations** (model methods). `{PK}` marks the primary key, `FK → X` marks a foreign key and its `on_delete` rule, and the crow's foot marks the "many" end of each relationship.

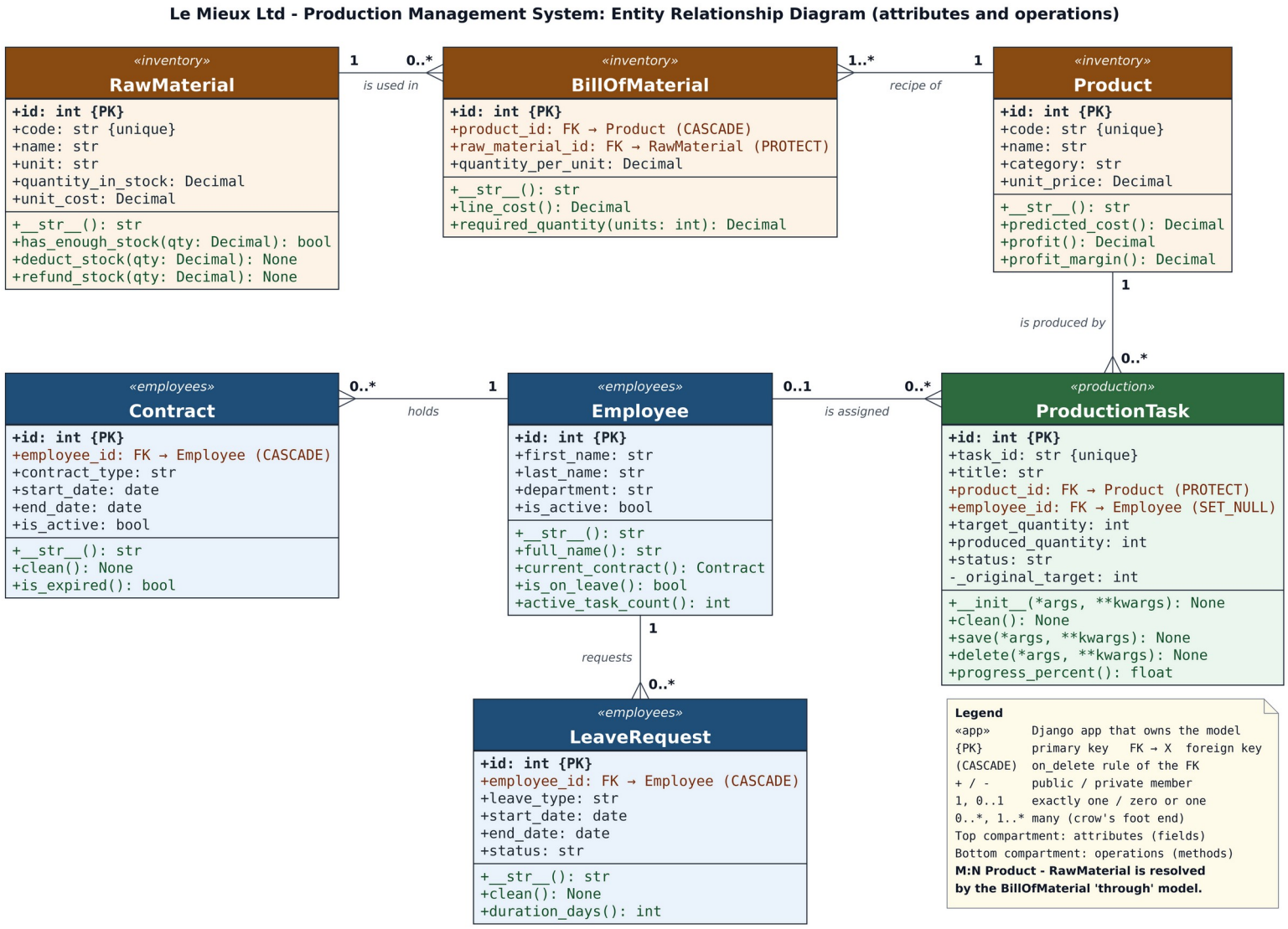


Figure 3: *Entity Relationship Diagram with attributes and operations (designed in Dia)*

3.3 Model Operations (Methods)

The operations in the ERD hold the business logic of the system. Keeping this logic inside the models (a "fat model" approach) means that the same rules apply whether data is changed from a form, from the dashboard or from the Django Admin site.

Table 6: *Operations of each model*

Model	Operation	Purpose

RawMaterial	`has_enough_stock(qty)`
	`deduct_stock(qty)`
	`refund_stock(qty)`
		Checks whether the warehouse holds a quantity; removes it from stock; returns it to stock.

Product	`predicted_cost()`
	`profit()`
	`profit_margin()`
		Sums the BOM line costs; subtracts the cost from `unit_price`; expresses the profit as a percentage of the price.

BillOfMaterial	`line_cost()` `required_quantity(units)`	Cost of one BOM line (`quantity_per_unit × unit_cost`); quantity needed to make a given number of units.

Employee	`full_name()`
	`current_contract()`
	`is_on_leave()`
	`active_task_count()`
		Display name; active contract; whether an approved leave covers today; number of Pending or In Progress tasks (used by the workload radar).

Contract	`clean()` `is_expired()`	Validates that the end date is not before the start date; tells whether the contract has ended.
LeaveRequest	`clean()` `duration_days()`	Validates the date range; returns the number of leave days.

ProductionTask	`__init__()`
	`clean()`
	`save()`
	`delete()`
	`progress_percent()`
		Stores `_original_target`; enforces the status rules and the stock check; deducts or refunds the stock difference; refunds all stock on delete; shows progress as a percentage.

All models	`__str__()`	Returns a readable name used in drop-down lists, tables and the Admin site.

3.4 Cardinality

Table 7: *Cardinality of the relationships*

Relationship	Cardinality	Explanation and implementation
Product – RawMaterial	M : N	A leather bag needs several raw materials (leather, thread, zippers), and zippers are used in several products. The relationship is resolved by the BillOfMaterial "through" model, which also stores the `quantity_per_unit` needed for the recipe.

Relationship	Cardinality	Explanation and implementation
Product – ProductionTask	1 : N	One product can be made in many production tasks; each task produces exactly one product.
Employee – ProductionTask	1 : N	An artisan can be assigned many tasks, but each task is assigned to at most one artisan. Implemented with a ForeignKey (`related_name='tasks'`).
Employee – Contract	1 : N	An employee can have a history of several contracts. Implemented with a ForeignKey (CASCADE).
Employee – LeaveRequest	1 : N	An employee can submit many leave requests. Implemented with a ForeignKey (CASCADE).

3.5 Referential Integrity (on_delete Rules)

Each foreign key has a deliberate deletion rule so that important history is never lost by accident:

Table 8: *Deletion rules and their effect*

Rule	Used on	Effect
CASCADE	BillOfMaterial → Product; Contract → Employee; LeaveRequest → Employee	Deleting the parent also deletes its dependent rows (a product's recipe, an employee's contracts and leave).
PROTECT	BillOfMaterial → RawMaterial; ProductionTask → Product	A raw material used in a recipe, or a product that has tasks, cannot be deleted.
SET_NULL	ProductionTask → Employee	If an employee is removed, their tasks are kept and simply become unassigned.

4. URL Routing and Views

Project-level URL patterns in `config/urls.py` delegate to each app's own `urls.py` using `include()`. Table 9 lists the main routes.

Table 9: *Main URL patterns and the views that handle them*

URL pattern	View	Purpose
`/`	`dashboard`	Aggregates database data to render the AdminLTE dashboard, workload radar and alerts.
`/inventory/products/<int:pk>/`	`product_detail`	Shows a product with its BOM, predicted cost and profit margin (dynamic URL parameter).
`/inventory/products/add/`	`product_create`	Creates a product together with its BOM lines using inline formsets.
`/employees/artisans/`	`employee_list`	Displays the HR roster, using `@property` methods to show live contract and leave status.
`/production/tasks/<int:pk>/edit/`	`task_update`	Edits task progress; deducts or refunds raw material stock when saved.
`/production/tasks/<int:pk>/delete/`	`task_delete`	Asks for confirmation, then deletes the task and refunds all its stock to the warehouse.

4.1 Request Cycle Example

Figure 4 follows a single request, `GET /production/tasks/5/edit/`, from the browser to the rendered page:

1. The browser sends the request.
2. `config/urls.py` matches the `production/` prefix and passes the rest of the path to `production.urls`.
3. `production/urls.py` matches `tasks/<int:pk>/edit/` and selects the `task_update` view with `pk=5`.
4. The `@login_required` decorator checks the user. If the user is not logged in, they are redirected to the login page.
5. The view loads the task with `get_object_or_404()` and fills a `TaskForm` (ModelForm) with it.
6. The view renders `production/task_form.html`, which extends `base.html` and its AdminLTE layout.
7. The finished HTML page is returned to the browser as the HTTP response.

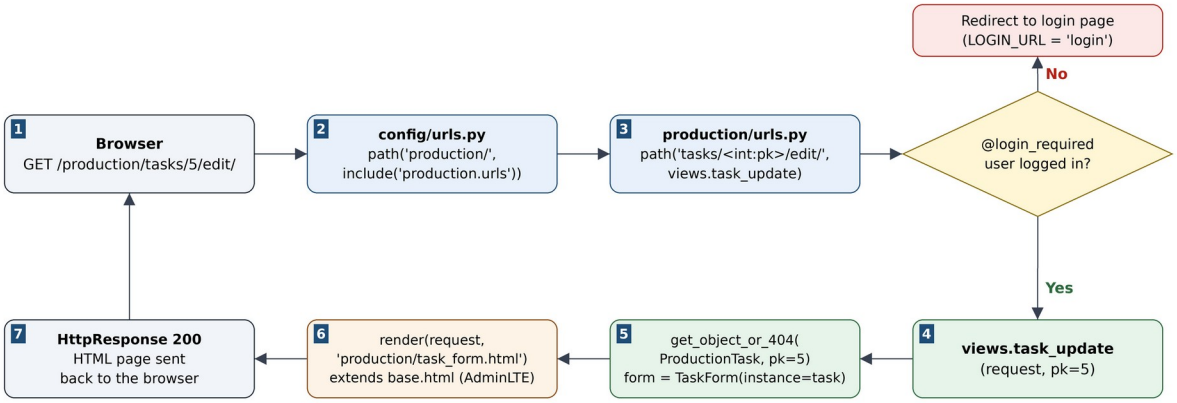


Figure 4: *Request cycle: from the URL to the rendered response*

5. Templates and AdminLTE Integration

AdminLTE is integrated through Django's template inheritance. Its files are served from the `static/` folder and loaded with the `{% static %}` tag inside the shared base template, so every page has the same look and navigation.

Table 10: *Template components and AdminLTE integration*

Component	Description
Base template (`base.html`)	Loads all CSS, JavaScript, fonts and plugins. Defines the shared layout: a sidebar menu and a top navigation bar with a "Quick Add" drop-down and the current user's role.
Dashboard (`dashboard.html`)	Extends `base.html`. Uses the Bootstrap grid (`col-lg-8` and `col-lg-4`) for summary cards, a recent-tasks table filled from the database and the Active Workload panel.
DRY principle	View, Edit and Delete actions look the same in every table, and all forms share one layout, giving a consistent user interface.
Dynamic data display	Records appear in tables (list pages) and cards (detail pages). The layout adapts to screen size through the Bootstrap grid.

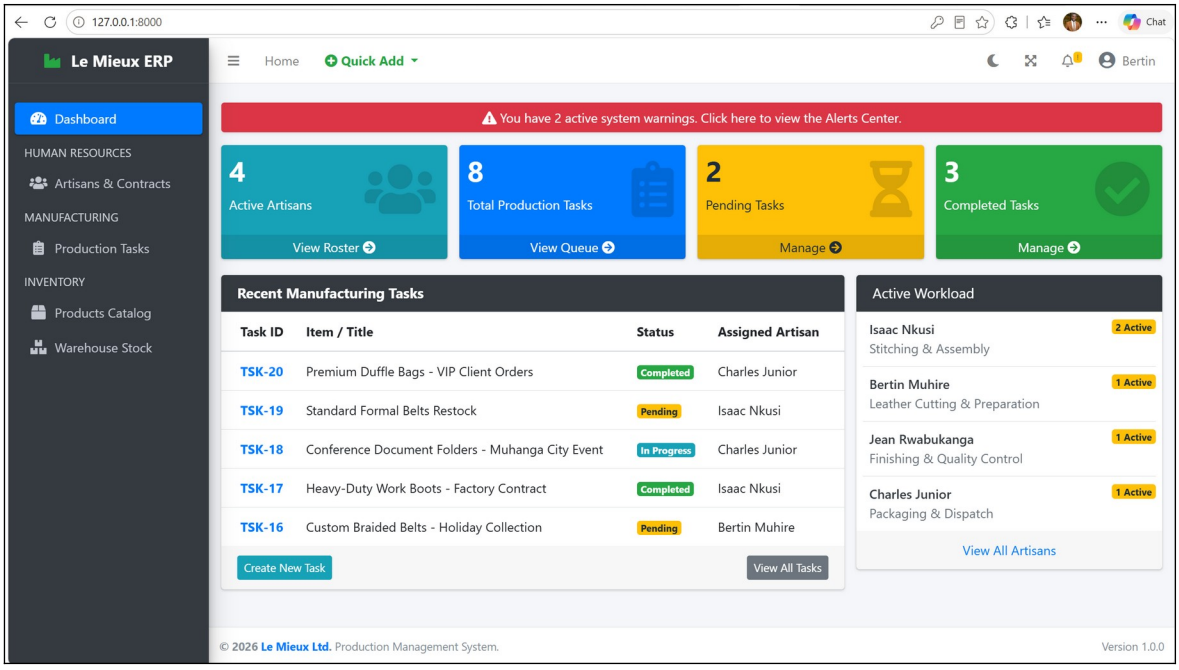


Figure 5: *AdminLTE dashboard: summary cards, recent tasks table and Active Workload radar*

6. CRUD Operations and Validation

All Create, Read, Update and Delete operations are built with Django Forms and ModelForms.

6.1 Core Operations

Table 11: *CRUD operations implemented*

Operation	Implementation
Create / Update	Uses shared form templates. `ProductForm` is combined with `inlineformset_factory` so that users can build the BOM recipe on the same page as the product (Figure 6).
Read	List views (tables) and detail views (cards) display records from the database (Figure 7).
Delete	Every delete goes through `confirm_delete.html`, so the user must confirm before a record is removed (Figure 8).

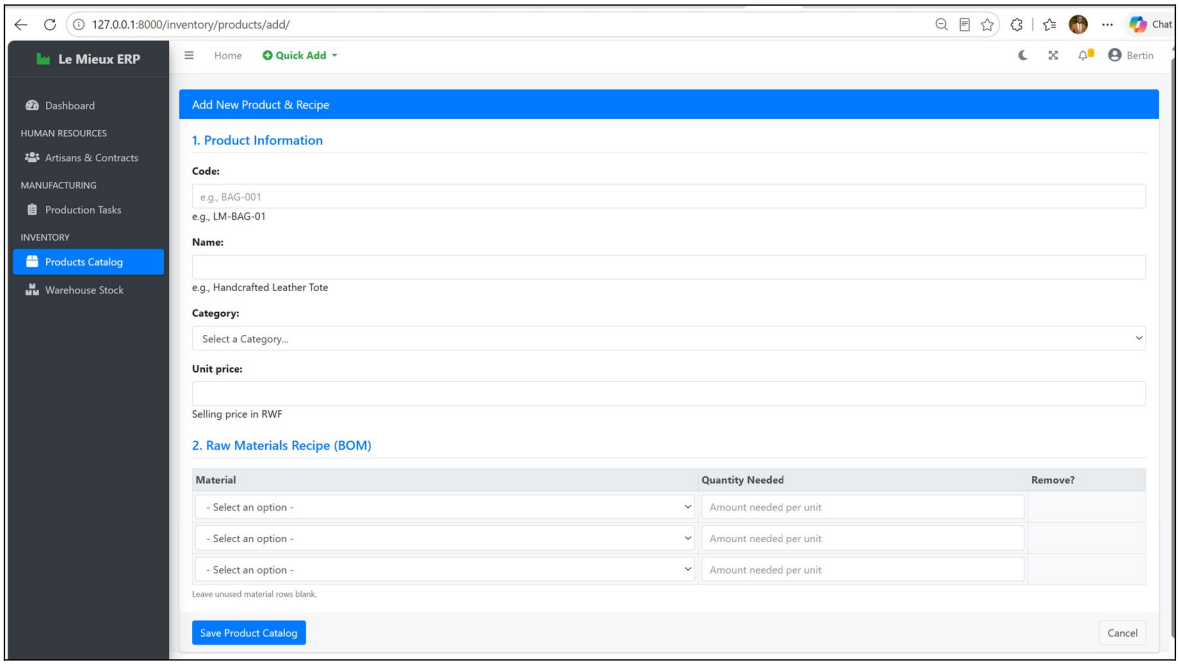


Figure 6: *Product creation form with the Bill of Materials (BOM) inline formset*

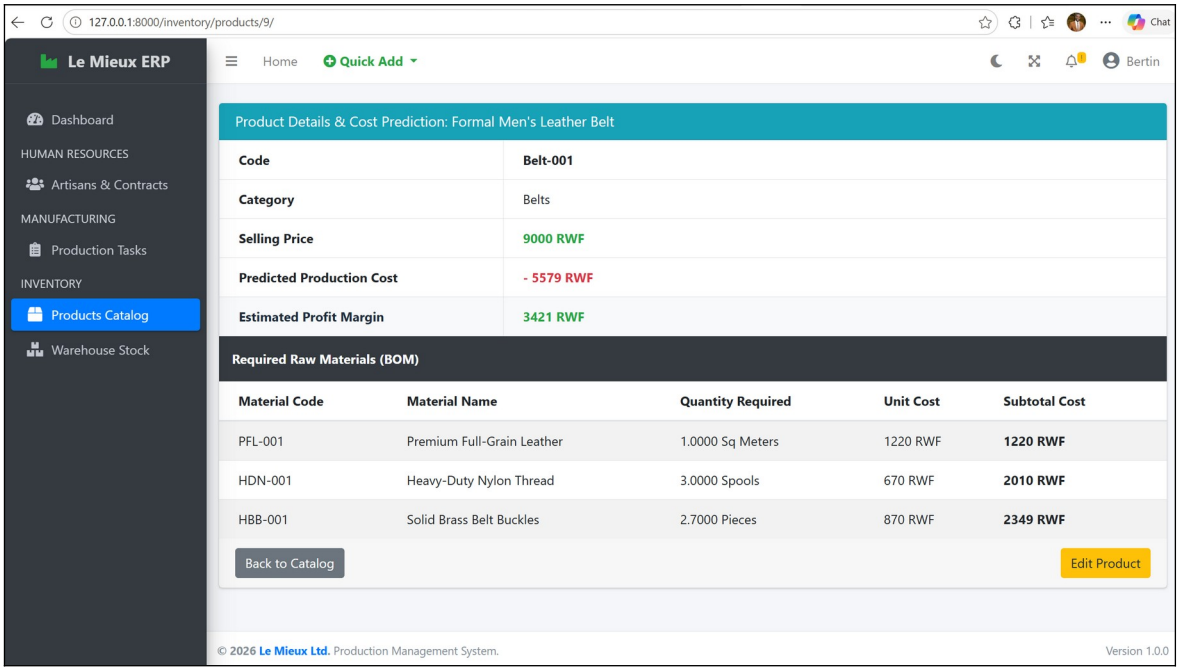


Figure 7: *Product detail page showing the BOM, predicted cost and profit margin*

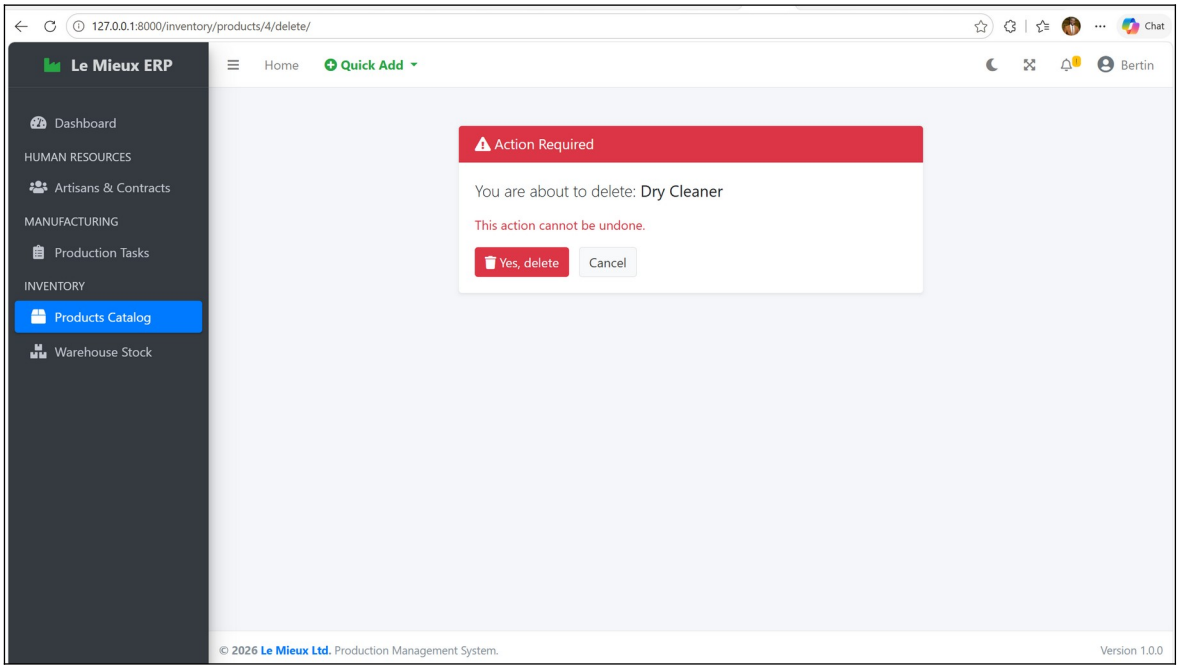


Figure 8: *Delete confirmation page (confirm_delete.html)*

6.2 Advanced Validation (clean() Overrides)

Business rules are enforced in Python by overriding the `clean()`, `save()` and `delete()` methods, as summarised in Table 12 and Figure 9.

Table 12: *Validation and stock rules*

Rule	Description
Status (state-machine) rule	A ProductionTask cannot be saved as "Pending" if any quantity has been produced, and cannot be "Completed" unless the produced quantity equals the target quantity.
Inventory protection rule	When a task is created, the form multiplies the requested quantity by each BOM line. If `quantity_in_stock` is too low for any material, a `ValidationError` stops the task from being created.
Automated stock calculation	The model remembers the task's `_original_target`. If a manager increases the target, the extra material is deducted; if the target is lowered or the task is deleted, the material is refunded.

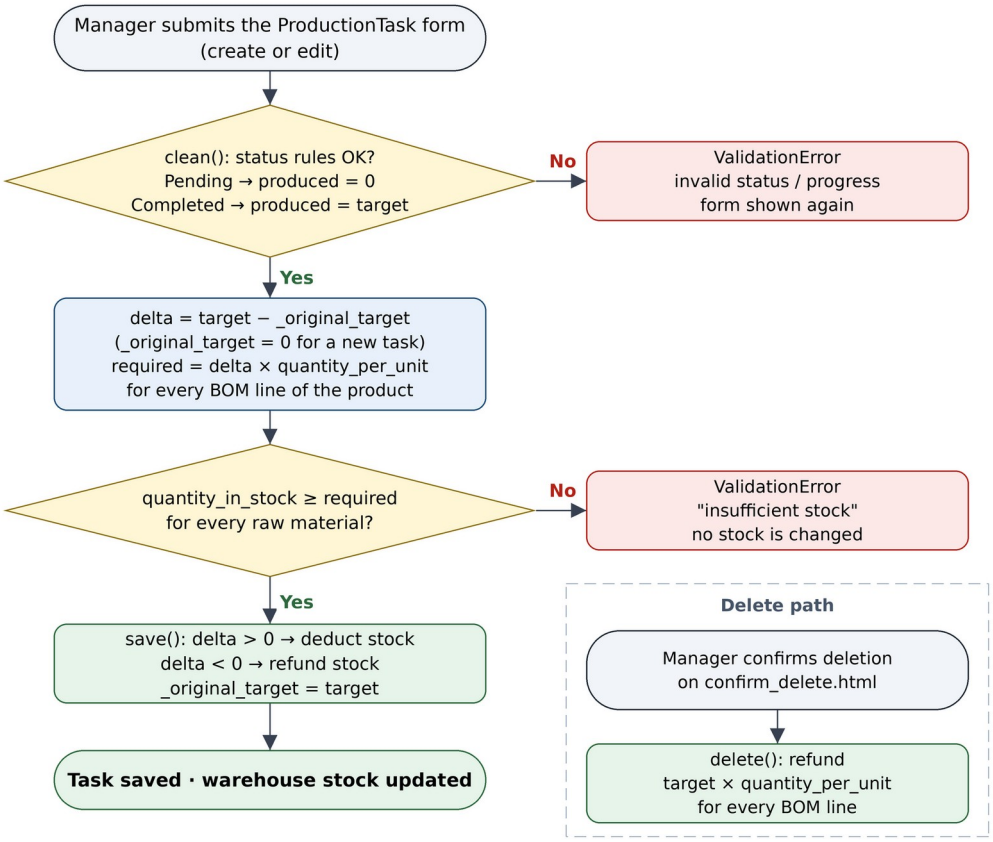


Figure 9: *Inventory protection and automatic stock deduction / refund logic*

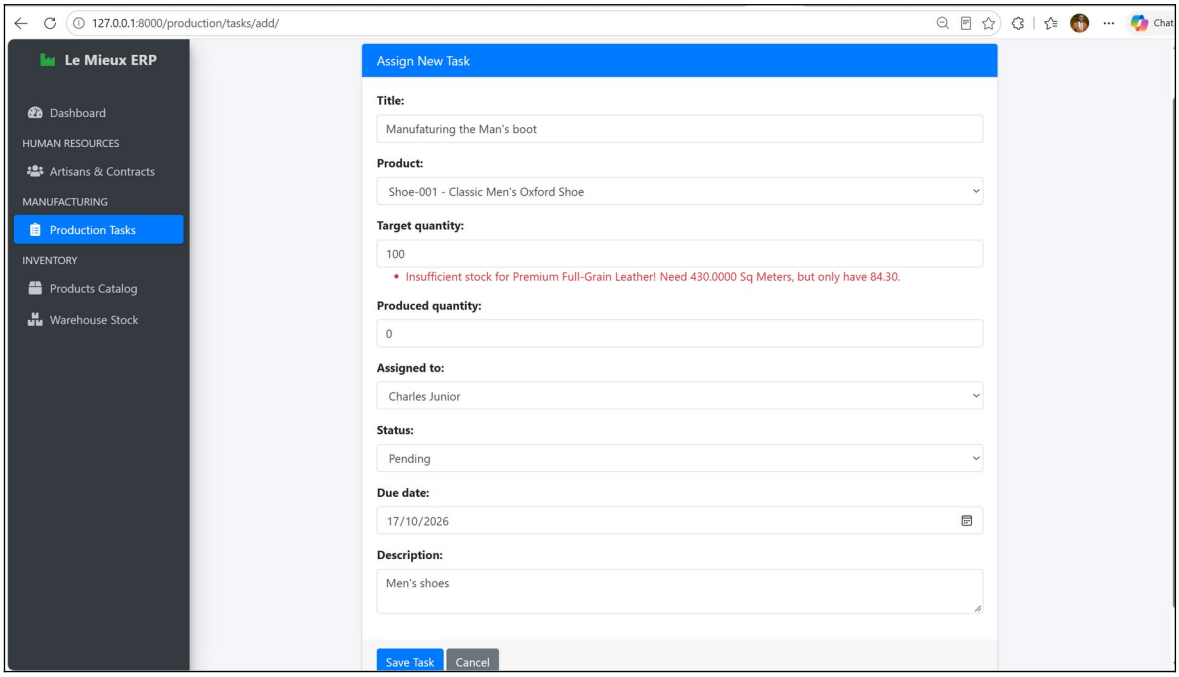


Figure 10: *Validation error shown when warehouse stock is insufficient for a new task*

7. Django Admin, Authentication and Authorisation

Table 13: *Django Admin and authentication features*

Feature	Implementation
Django Admin	All models are registered in `admin.py` with custom `list_display`, `search_fields` and `list_filter`, so every record can be managed from the built-in Admin site (`/admin/`).
Superuser	A superuser account (`admin`) was created to oversee all factory operations.
Authentication	Django's built-in `LoginView` and `LogoutView` handle sign-in and sign-out, with `LOGIN_URL = 'login'` in `settings.py`.
Authorisation	Create, update and delete pages are protected with `@login_required` and `@user_passes_test`, so only HR and production staff can change records. Artisans have view-only access.

Table 14: *Demonstration superuser account*

Item	Value
Admin URL	http://127.0.0.1:8000/admin/
Username	admin
Password	Admin@2026

Note: these credentials are for the local demonstration only and must be changed before any real deployment.

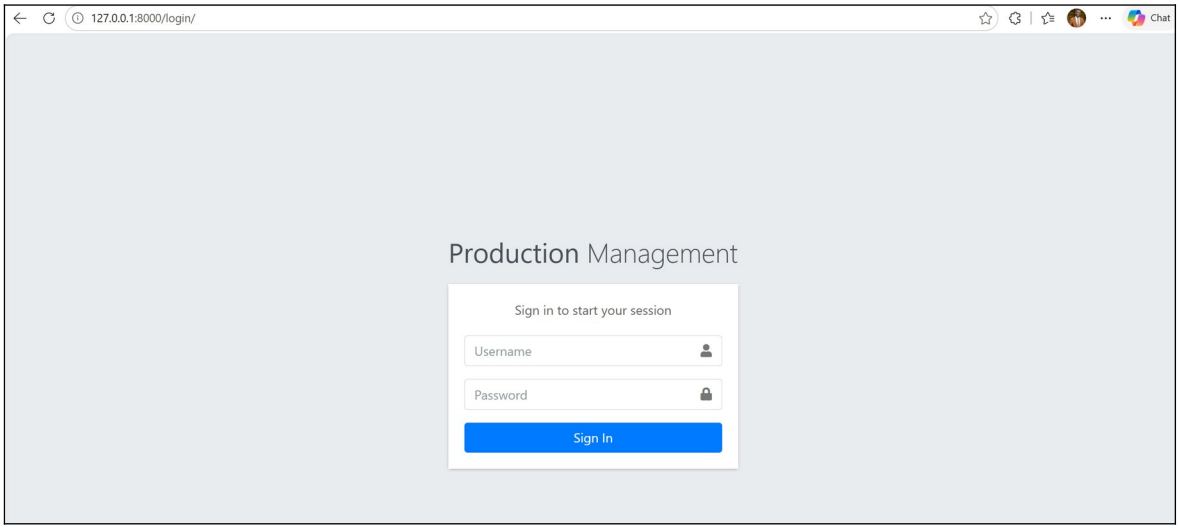


Figure 11: *Login page of the application*

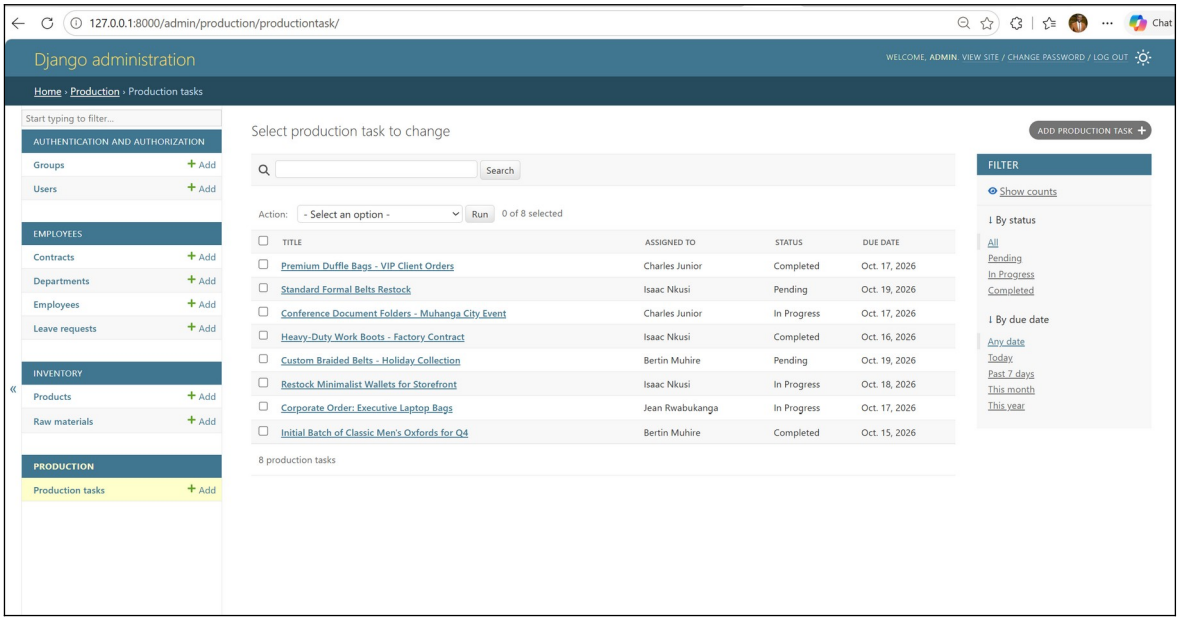


Figure 12: *Django Admin site: registered models with list display, search and filters*

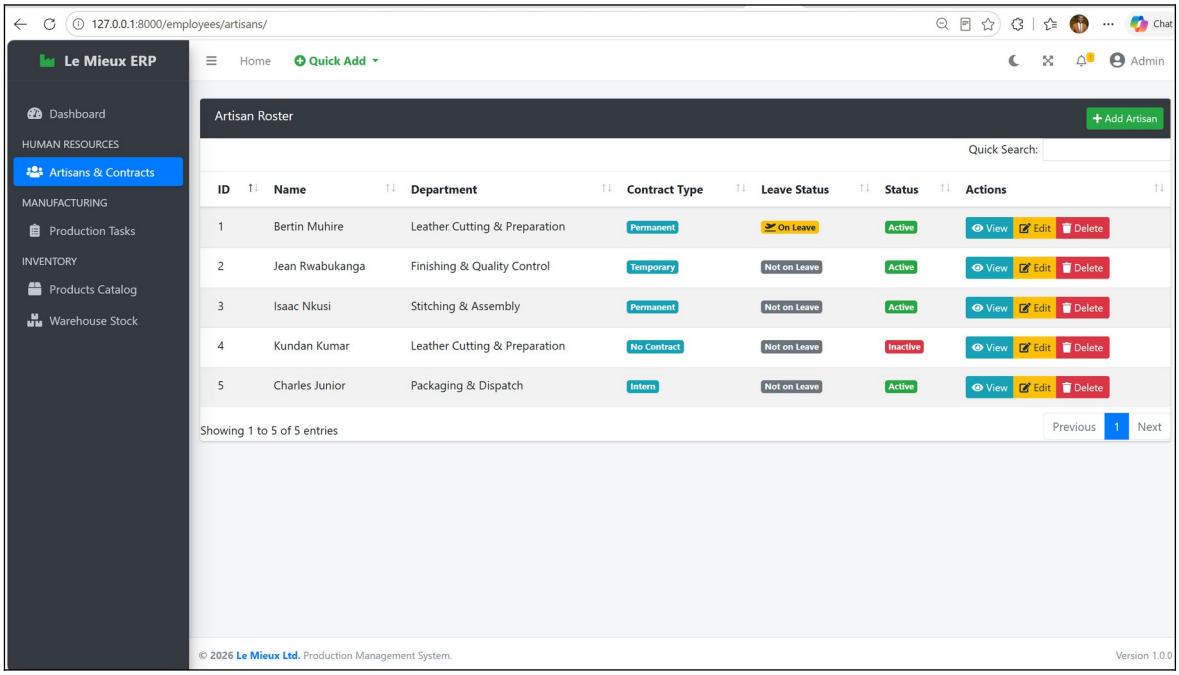


Figure 13: *HR roster of artisans with live contract and leave status*

8. Challenges and Solutions

Table 15: *Main challenges and how they were solved*

Challenge	Problem	Solution
Creating the BOM recipe	Raw materials had to be assigned while the product itself was being created.	Used `inlineformset_factory(Product, BillOfMaterial, form=BOMForm)`, so the parent and child records are saved together from one page.
Dashboard query performance	Counting each artisan's active tasks one by one would send many queries to the database.	Used one annotated query with `Count` and `Q`: `annotate(active_tasks=Count('tasks', filter=Q(tasks__status__in=['Pending', 'In Progress'])))`
Accurate stock deductions	Editing a task could deduct the same stock twice.	Overrode the model's `__init__()` to remember `_original_target`, so `save()` deducts or refunds only the difference.
Leave-request validation crash	Validation failed because the Employee foreign key was not yet attached to the form.	Set `form.instance.employee = employee` before calling `form.is_valid()`.

9. Conclusion and Future Work

The Production Management System was implemented successfully. It meets the structural requirements of the project (three apps and seven models), integrates the AdminLTE dashboard, predicts production cost from the Bill of Materials and protects the warehouse by deducting and refunding stock automatically. Together, these features replace scattered paper and spreadsheet records with one reliable tool for Le Mieux Ltd.

Possible improvements for the next version include:

- A REST API (for example with Django REST Framework) so that other systems or a mobile app can use the data.
- Low-stock alerts and purchase suggestions for raw materials.
- Downloadable PDF or Excel reports on production, costs and leave.
- Deployment on a production server with a client-server database such as PostgreSQL.

Appendix A: How to Run the Project

1. Extract the submitted ZIP file.
2. Open a terminal in the project's root folder, then create and activate a virtual environment:

```
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate     # macOS / Linux
```

3. Install the dependencies:

```
pip install -r requirements.txt
```

4. Create the SQLite database with migrations:

```
python manage.py makemigrations
python manage.py migrate
```

5. Start the development server:

```
python manage.py runserver
```

6. Open the dashboard at http://127.0.0.1:8000/ and the Django Admin site at http://127.0.0.1:8000/admin/.

Appendix B: Diagram Source Files

All diagrams in this report were designed in **Dia**. The editable source files are submitted with the report:

Table 16: *Dia source files*

Figure	Diagram	Dia file
Figure 1	Use case diagram	`Fig1_Use_Case_Diagram.dia`
Figure 2	System architecture	`Fig2_System_Architecture.dia`
Figure 3	ERD with attributes and operations	`Fig3_ERD_Production_Management_System.dia`
Figure 4	Request cycle	`Fig4_Request_Cycle.dia`
Figure 9	Stock validation flowchart	`Fig9_Stock_Validation_Flowchart.dia`